

QUESTION 47

Aim

To find an approximate vertex cover of a graph using a greedy approximation algorithm by selecting uncovered edges and adding their endpoints into the cover set.

Problem Statement

Given an un-directed graph, find a set of vertices such that every edge of the graph is connected to at least one selected vertex.

The algorithm should repeatedly select an uncovered edge, add both endpoints of that edge to the vertex cover, and continue until all edges are covered.

Input: Graph with vertices and edges.

Output: Approximate vertex cover set.

Introduction

A Vertex Cover is a set of vertices in a graph where every edge has at least one endpoint in the selected set.

Finding the minimum vertex cover is an NP-Hard problem. For large graphs, finding the exact solution is time-consuming. Approximation algorithms are used to find a valid solution quickly.

In this approach, an uncovered edge is selected and both its endpoints are added to the cover. The selected vertices cover that edge and other connected edges. The process continues until all edges are covered.

1. Objectives

2. Find a valid vertex cover for a graph.
3. Use approximation technique for faster solution.
4. Cover all edges of the graph.
5. Implement greedy selection method.
6. Analyze time and space complexity

Constraints

- Every edge must be covered.
- The selected vertices should form a valid cover.
- All edges must be checked.
- The algorithm should avoid unnecessary operations.

Algorithm

Step 1:

Input the graph containing vertices and edges.

Step 2:

Initialize an empty vertex cover set.

Step 3:

Select an uncovered edge (u, v) .

Step 4:

Add both vertices u and v into the cover set.

Step 5:

Remove all edges covered by these vertices.

Step 6:

Repeat the process until no edges remain.

Step 7:

Display the vertex cover set.

Pseudocode

Algorithm VertexCover(G)

Input:

Graph $G(V, E)$

Output:

Vertex Cover Set C

Begin

C = empty set

While edges are remaining do

 Select an uncovered edge (u, v)

 Add u and v to C

 Remove all edges connected to u or v

End While

Return C

End

Source Code

Vertex Cover Approximation

```

edges = [(1,2), (2,3), (3,4), (4,1)]

cover = set()

while edges:

    u, v = edges[0]

    cover.add(u)
    cover.add(v)

    edges = [e for e in edges if u not in e and v not in e]

print("Vertex Cover:", cover)

```

Sample Output

Vertex Cover:
{1,2,3,4}

Working Example

Graph:

```

1 ---- 2
|      |
|      |
4 ---- 3

```

Edges:

(1,2), (2,3), (3,4), (4,1)

Select edge (1,2)

Add:

Cover = {1,2}

Select remaining edge (3,4)

Add:

Cover = {1,2,3,4}

All edges are covered.

Complexity Analysis

Time Complexity

$O(E)$

Where E is the number of edges. Each edge is processed during the algorithm.

Space Complexity

$O(V)$

Where V is the number of vertices stored in the cover set.

Applications

- Computer networks.
- Resource allocation.
- Security monitoring.
- Scheduling systems.
- Database optimization.
- Graph-based optimization

Result

The Vertex Cover Approximation Algorithm was successfully implemented. The algorithm selected uncovered edges and added their endpoints to the cover set. It produced a valid approximate vertex cover for the given graph.